Clinical GraphRAG for Mental Health Question Answering: A Graph-Grounded, Safety-Aware Retrieval-Augmented Framework

Ecole Supérieure en Sciences et Technologies de l'Informatique et du Numérique — Béjaïa

April 23, 2026

Abstract

Large Language Models (LLMs) are increasingly used for medical information support, yet hallucinated or weakly grounded outputs remain a critical risk in healthcare contexts. This paper presents and evaluates a clinical GraphRAG framework for mental health question answering with explicit evidence grounding and safety-aware routing. The proposed pipeline constructs a domain knowledge graph from authoritative sources (WHO, NIMH, and DSM-5-derived material) through reproducible stages: source ingestion, markdown-based segmentation, contextual enrichment, schema-constrained entity–relation extraction over a fixed ontology of 11 entity types, 13 relation types, and 25 valid triplet patterns, followed by graph cleaning, targeted repair, and Leiden-based community summarization.

At inference time, the system applies a safety-first decision layer for crisis detection, then performs graph-first retrieval for in-scope questions using condition lookup, reverse symptom lookup, comparison reasoning, and entity-centric relation exploration. Responses include both chunk-level citations and source payloads to support traceability and manual verification.

We evaluate three retrieval configurations—vector baseline, cross-encoder reranked vector, and the full GraphRAG pipeline—using both RAGAS metrics and deterministic proxy metrics. The resulting framework provides an end-to-end, auditable, and reproducible workflow for grounded clinical question answering.

1 Introduction

Recent LLM systems can generate fluent medical answers, but fluency is not equivalent to correctness [7]. In healthcare settings, unsupported statements can mislead users about symptoms, treatments, urgency, and risk, especially when users treat conversational responses as practical guidance. This makes factual grounding and safety behavior first-class system requirements.

Retrieval-Augmented Generation (RAG) [5, 8] addresses part of this challenge by conditioning generation on external evidence. However, in clinical domains, many questions

are relational rather than purely semantic: users ask about condition–symptom links, differential diagnosis, prevalence statements, treatment suitability, and safety signals. Standard vector-based RAG often fails to capture these multi-hop clinical relationships [12]. A knowledge graph representation is well suited to these relation-centric queries because it explicitly stores typed entities and typed edges [6].

Graph-based RAG approaches, or GraphRAG [3], extend traditional RAG by structuring retrieved evidence as graph-based community summaries rather than flat document chunks. This paper documents and evaluates an implemented Clinical GraphRAG system centered on mental health question answering. The system includes reproducible data ingestion, checkpointed graph construction, schema-constrained extraction, graph cleaning and repair stages, safety gating, and a query interface for graph-grounded answering.

The practical goals of this work are:

- reduce unsupported generations by constraining responses to retrieved graph evidence,
- preserve provenance through chunk-level citations and source metadata,
- improve robustness with targeted repair passes and regression testing,
- enforce a safety short-circuit for crisis-like queries,
- provide a reproducible, checkpoint-backed pipeline for clinical knowledge graph construction.

This manuscript focuses on the engineering and design of an auditable RAG pipeline and its observed system behavior, while keeping claims aligned with what is currently implemented and evaluated.

2 Related Work

Retrieval-Augmented Generation. Lewis et al. [8] introduced RAG, combining a parametric language model with a non-parametric retrieval module to ground generation in external documents. REALM [5] further explored pre-training with retrieval. While effective for open-domain QA, these approaches rely on vector similarity over flat text chunks, which can miss relational and multi-hop reasoning patterns common in clinical queries.

Knowledge Graphs for Clinical NLP. Knowledge graphs provide structured, queryable representations of domain knowledge [6]. In medicine, KGs have been used to organize symptoms, treatments, and diagnostic criteria, enabling relation-centric reasoning that complements LLM fluency [12]. However, constructing clinically accurate KGs at scale remains challenging due to extraction noise, schema enforcement, and maintenance of provenance.

GraphRAG. Edge et al. [3] proposed GraphRAG, which partitions a knowledge graph into communities and generates summaries to support global queries. Our work builds on this paradigm, extending it with a clinical ontology, schema-constrained extraction, and safety-aware query routing.

Medical LLMs. Large-scale clinical models such as Med-PaLM [14] and GPT-4-based medical systems [11] demonstrate strong clinical QA performance, but remain susceptible to hallucination. Our approach constrains generation to graph-retrieved evidence, providing an orthogonal mechanism for reducing unsupported claims.

3 Dataset Construction and Processing

3.1 Data Sources

We construct a domain-specific corpus from authoritative public sources in the mental health domain:

- **World Health Organization (WHO):** mental health fact sheets, condition guides, and treatment recommendations [16].
- **National Institute of Mental Health (NIMH):** condition-specific pages covering symptoms, risk factors, treatments, and prevalence data [9].
- **DSM-5-derived material:** diagnostic criteria, specifiers, course features, and differential diagnosis content, filtered from structured reference material [1].

Source definitions are maintained in a JSON registry (`source_registry.json`) and converted into a provenance manifest that records authority, condition tag, URL, parser mode, and output markdown path. This manifest enables full traceability from any graph entity back to its source document.

3.2 Data Extraction and Contextual Enrichment

The ingestion layer uses format-specific extraction strategies:

- **HTML sources** are converted using `trafilatura` [2], a dedicated web text extraction library.
- **PDF sources** are converted using `pymupdf4llm` with layout-aware extraction; a fallback to legacy extraction mode engages automatically when layout parsing fails.

After ingestion, markdown documents are parsed into structural nodes using header-based segmentation rather than arbitrary fixed-length splitting. Each node preserves source metadata (authority, condition, source label, section path, and file path), enabling citation and auditing at the chunk level.

For contextual enrichment, an LLM generates a short context sentence per node (stored as a `context_tag` and prepended to node text). This enrichment stage is checkpointed and resumable: interrupted runs can continue deterministically under compatible node hash, model, and prompt version.

3.3 Preprocessing and Quality Control

Preprocessing is designed for quality control before graph extraction:

- **DSM filtering:** a two-layer filter removes non-clinical administrative sections and low-value chunks based on length, table/list heaviness, alpha ratio, and clinical signal presence.
- **Node integrity checks:** fingerprinting and checkpoint validation prevent stale cache reuse.
- **Citation normalization:** standardized source labels and section-aware citation strings are generated at ingestion time.
- **Resumable enrichment:** context-tag generation with periodic checkpoint snapshots and metadata compatibility checks (node hash, prompt version, model name).

4 Knowledge Graph Construction

4.1 Graph Formalization

We formalize the clinical knowledge graph as $\mathcal{G} = (V, E, \Phi)$, where V is the set of typed clinical entities, E is a set of directed, typed edges, and Φ encodes node and edge attributes including provenance metadata. Each edge is represented as:

$$\mathcal{T} = (s, r, o, \mathcal{C}_{\mathcal{T}}, \sigma) \tag{1}$$

where $s, o \in V$ are entities, r is a clinical relationship from the allowed relation set, $\mathcal{C}_{\mathcal{T}}$ is the set of source chunk citations supporting this triple, and σ denotes the source authority (WHO, NIMH, or DSM-5). This representation enables symbolic traceback for every generated claim.

4.2 Entity and Relation Typing

The graph schema is constrained to a fixed ontology of 11 entity types and 13 relation types. Entities are categorized as shown in Table 1.

Table 1: Allowed entity types in the clinical knowledge graph.

Entity Type	**Example**	**Description**
CONDITION	Major Depressive Disorder	Mental health disorder
SYMPTOM	Anhedonia	Observable clinical sign
TREATMENT	Cognitive Behavioral Therapy	Therapeutic intervention
MEDICATION	SSRIs	Pharmacological agent
SAFETY_RESOURCE	988 Lifeline	Crisis support resource
DEMOGRAPHIC	Children, Elderly	Population group
DIAGNOSTIC_CRITERION	Criterion A	DSM-5 criterion
SPECIFIER	With anxious distress	Disorder qualifier
RISK_FACTOR	Family history	Predisposing factor
COURSE_FEATURE	Chronic, Episodic	Temporal progression
PREVALENCE_STATEMENT	8.4% of U.S. adults	Population statistic

Relations encode clinically validated interactions as shown in Table 2.

Beyond entity and relation type constraints, the system enforces 25 valid triplet patterns—explicit $(subject_type, relation, object_type)$ combinations. Any extracted triple not matching a valid pattern is rejected during extraction and cleaning.

4.3 Schema-Constrained Extraction

Graph extraction proceeds per enriched node using an LLM with a structured extraction prompt that specifies:

1. the allowed entity types and relation types,
2. valid triplet patterns with examples,
3. output format (JSON with entities and relations arrays),
4. instructions to avoid placeholders, vague names, and duplicate entities.

Table 2: Allowed relation types and representative triplet patterns.

Relation	Source → Target	Example
HAS_SYMPTOM	Condition → Symptom	Depression → Anhedonia
TREATED_BY	Condition → Treatment/Medication	GAD → CBT
PRESCRIBES	Condition → Medication	MDD → SSRIs
SUITABLE_FOR	Treatment/Med. → Demographic	CBT → Adolescents
CONTRAINDICATED_FOR	Treatment/Med. → Demographic	TCA → Elderly
URGENT_ACTION	Condition/Symptom → Safety_Res.	Suicidal ideation → 988
HAS_DIAG._CRITERION	Condition → Diagnostic_Criterion	PTSD → Criterion B
HAS_SPECIFIER	Condition → Specifier	MDD → Anxious distress
HAS_RISK_FACTOR	Condition → Risk_Factor/Demo.	MDD → Family history
DIFFERENTIAL_DIAG.	Condition → Condition	GAD → PTSD
COMORBID_WITH	Condition → Condition	Depression → Anxiety
HAS_COURSE	Condition → Course_Feature	Schizophrenia → Chronic
HAS_PREVALENCE	Condition → Prevalence_Stmt.	Depression → 8.4%

The extraction function parses the LLM's JSON output with fallback extraction when responses are fenced or noisy. Extraction quality is tracked via `GraphExtractionStats`, recording per-node success rates, entity/relation counts, and error categories. The process supports partial-state checkpointing, allowing interrupted long runs to resume without re-extracting completed nodes.

4.4 Normalization and Cleaning

Post-extraction, multiple quality layers refine the graph:

Canonicalization. Entity names are normalized to uppercase canonical forms. An abbreviation map (19 clinical abbreviations, e.g., CBT → COGNITIVE BEHAVIORAL THERAPY, PTSD → POST TRAUMATIC STRESS DISORDER) expands common shorthand. Equivalent entities are merged by canonical form, consolidating their relations, descriptions, and source provenance.

Garbage and placeholder filtering. Rule-based filters remove entities with generic names (e.g., "PATIENT", "DISORDER", "THE INDIVIDUAL"), overly short names, or non-alphabetic content. Placeholder entity names (e.g., "CONDITION X", "TREATMENT A") are detected and discarded.

Low-signal pruning. Entities with minimal descriptive content and no unique relations are pruned. Isolated nodes (no incident edges after cleaning) are removed.

Strict relation validation. Post-cleanup, each remaining relation is re-checked against the valid triplet type set, removing any edges that survived extraction but violate schema constraints.

4.5 Targeted Repair

Automated extraction can miss or under-represent clinically important facts. The system addresses this through two mechanisms:

LLM-based repair. For selected condition–relation targets (e.g., depression prevalence, GAD differential diagnosis, autism treatments), the system selects the most rel-

evant enriched nodes using a relevance scoring function, then prompts the LLM with a targeted repair prompt. Extracted entities and relations are applied to the existing graph state.

Curated payload injection. For critical factual gaps that require exact clinical content, curated payloads inject high-priority facts deterministically. Current curated targets include:

- Depression: prevalence data,
- Generalized Anxiety Disorder: prevalence and differential diagnosis,
- Post-Traumatic Stress Disorder: differential diagnosis,
- Autism Spectrum Disorder: treatment coverage.

After repair and injection, the graph undergoes a final cleaning pass and citation refresh to ensure consistency.

4.6 Community Detection and Summarization

The graph is partitioned into communities using the Leiden algorithm [15] (via the `graspologic` library, with random seed 42 for reproducibility). Each community groups clinically related entities, forming a higher-level organizational structure.

Community summaries are generated in two modes:

- **LLM-generated:** when a summary LLM client is configured, each community is summarized based on its member entities and internal relations.
- **Heuristic fallback:** when no LLM is available, summaries are constructed from member entity labels and relation patterns.

These summaries serve as global context during query-time retrieval, enabling the system to provide broader contextual information beyond direct entity lookups.

5 Problem Formulation

5.1 Task Definition

Let q denote a user query in the mental health domain, and let h denote optional session history for follow-up context. The system must return a response that is both clinically useful and auditable.

We define the returned payload as:

$$R = (A, \mathcal{C}, \mathcal{S}, t, b) \tag{2}$$

where A is the answer text, $\mathcal{C}$ is the citation list, $\mathcal{S}$ is the set of supporting source chunks (each containing chunk ID, citation string, text, section title, context tag, source label, file path, and header path), t is the classified query type, and b is a safety/status block (crisis flag, out-of-scope flag, matched safety indicators, and safety resources).

The answer is computed as:

$$A = f(q, h, \mathcal{G}, \Sigma) \tag{3}$$

where $\mathcal{G}$ is the graph artifact store (entities, relations, provenance maps) and Σ is the set of community summaries.

5.2 Retrieval Mechanism

Before retrieval, a safety and domain classifier routes the request into one of three classes:

$$\text{class}(q) \in \{\textsc{Crisis}, \textsc{Out-of-Scope}, \textsc{In-Scope}\} \tag{4}$$

For in-scope queries, the system applies graph-first multi-path retrieval. The router selects among relation-aware branches based on query intent classification:

- **Forward lookup:** condition-centric relation expansion (e.g., "What are the symptoms of depression?"),
- **Reverse symptom lookup:** symptom-to-condition ranking (e.g., "What conditions cause fatigue and insomnia?"),
- **Comparison lookup:** shared and diverging relations between two conditions (e.g., "How does GAD differ from PTSD?"),
- **Entity lookup:** incoming and outgoing relation view for non-condition entities,
- **Community context:** matched community summaries for global contextual grounding.

The runtime can optionally synthesize a final answer with an LLM, but only after graph evidence is assembled. The LLM receives bounded, structured context blocks—it does not generate from parametric knowledge alone.

5.3 Hallucination Reduction Objective

We define hallucination as any answer claim not supported by retrieved graph facts, relation metadata, or attached source chunk evidence. Let $\mathcal{U}(A)$ be the set of atomic claims in answer A, and let $\mathcal{E}$ be the retrieved evidence set. A claim $u \in \mathcal{U}(A)$ is grounded if it is entailed by some $e \in \mathcal{E}$.

The practical objective is:

$$\max \ \text{Accuracy}(A) \quad \text{and} \quad \min \ \mathcal{H}(A) \tag{5}$$

with the hallucination rate defined as:

$$\mathcal{H}(A) = 1 - \frac{|\mathcal{U}_{\text{grounded}}(A)|}{|\mathcal{U}(A)|} \tag{6}$$

The implemented system reduces $\mathcal{H}(A)$ through:

1. schema-constrained extraction and strict triplet validation at build time,
2. post-extraction cleaning, normalization, and targeted repair,
3. graph-first retrieval with explicit citation and source-chunk return at query time,
4. fallback behavior when graph evidence is insufficient,
5. crisis short-circuit logic that prevents normal QA for high-risk inputs.

6 System Architecture

6.1 Pipeline Stages

The end-to-end pipeline is orchestrated through five stages, each checkpointed for reproducibility and resumability:

1. **Source ingestion:** acquire and convert clinical sources to markdown with provenance manifest.
2. **Node parsing and enrichment:** segment markdown into structural nodes, attach source metadata, generate LLM context tags.
3. **Graph extraction:** schema-constrained entity–relation extraction with partial-state checkpointing.
4. **Cleaning and repair:** garbage filtering, normalization, merge of equivalent entities, targeted repair, curated payload injection.
5. **Community detection:** Leiden partitioning and community summary generation.

Each stage produces checkpointed artifacts (pickled graph state, metadata files) that can be inspected, resumed, or selectively rebuilt. The pipeline fails explicitly when an LLM is required but unavailable, rather than silently degrading.

6.2 Query Engine

At query time, the system loads the checkpointed graph artifacts—entities, relations, provenance maps, and community summaries—into an in-memory graph store. The query engine orchestrates the full retrieval and answer construction pipeline:

Query Processing Flow:

User Query → Safety Assessment → Query Classification
↪ CRISIS: immediate crisis response with resources
↪ OUT-OF-SCOPE: domain boundary response
↪ IN-SCOPE: Intent Routing → Graph Retrieval → Community Context
→ Citation Assembly → Optional LLM Synthesis → Structured Response

Figure 1: Query processing flow. In-scope queries are routed through intent-specific graph retrieval branches before answer construction.

The graph store supports four core query operations: *forward lookup* (outgoing relations from a condition), *entity relation lookup* (bidirectional relations for any entity), *reverse symptom lookup* (conditions matching given symptoms), and *text search* (keyword-based entity matching over names and descriptions). These operations are composed by the query router to serve different query intents through a unified orchestration function.

Session history can optionally be maintained to support follow-up questions (e.g., "What about its treatment?" after asking about depression symptoms), where prior questions are prepended to the current query for context during classification and routing.

7 Safety Shield

Before any graph retrieval, a rule-based safety assessment evaluates crisis risk. The safety module uses three categories of regex-based indicators:

1. **Explicit self-harm intent** (18 patterns): detects direct expressions of suicidal ideation or self-harm (e.g., "kill myself", "end my life", "suicidal", "self-harm").
2. **Imminence or plan** (13 patterns): detects temporal urgency or method mention (e.g., "tonight", "right now", "plan to", "overdose").
3. **Severe distress** (13 patterns): detects extreme emotional states (e.g., "hopeless", "worthless", "no reason to live", "hit rock bottom").

The module applies suppression logic for purely informational queries (e.g., "What is suicide prevention?") and elevation logic when first-person language co-occurs with crisis signals.

When a crisis is detected:

- graph retrieval is completely bypassed,
- the response includes emergency contact resources (988 Suicide & Crisis Lifeline, Crisis Text Line, Emergency Services),
- matched indicator labels are returned for transparency.

This layer is a practical guardrail, not a clinical triage system. It prioritizes recall over precision: false positives are acceptable, but false negatives in crisis detection are not.

8 Query Processing Pipeline

8.1 Query Classification

Each incoming query is classified through a multi-stage process:

1. **Safety assessment:** the safety shield evaluates crisis indicators. If triggered, the query is routed to crisis response immediately.
2. **Domain scope check:** keyword and heuristic analysis determines whether the query falls within the mental health domain.
3. **Intent detection:** for in-scope queries, the system identifies comparison intent (via patterns like "difference between X and Y"), symptom-based phrasing (via natural language heuristics and a 22-entry symptom phrase alias table mapping colloquial expressions to graph entities), and relation-specific intent (e.g., prevalence, treatment, diagnostic criteria).
4. **Entity matching:** the query is matched against graph entities using exact match, alias expansion, and fuzzy matching to identify referenced conditions, symptoms, and other entities.

8.2 Multi-Path Retrieval

Based on classification results, the system routes to one of several retrieval branches:

Comparison branch. Activated when two or more conditions are detected. Performs parallel forward lookups for each condition and computes shared and diverging relations, presenting a structured comparison.

Reverse symptom branch. Activated when symptom-like phrasing is detected without a specific condition. Matches mentioned symptoms against graph entities (including alias resolution) and ranks conditions by symptom overlap.

Forward lookup branch. The default for recognized conditions. Expands outgoing relations from the matched condition, optionally filtered by detected relation intent (e.g., only HAS_SYMPTOM edges for symptom queries).

Entity lookup branch. For non-condition entities (e.g., specific treatments or medications). Returns both incoming and outgoing relations to provide a complete relational view.

Community context. Optionally supplements any branch with matched community summaries for broader contextual grounding.

8.3 Answer Construction and Provenance

After graph retrieval, the system constructs a response with full provenance:

1. **Citation assembly:** chunk IDs from retrieved relations and entities are resolved to human-readable citation strings (format: *Source Authority — Document Title — Section*).
2. **Source chunk payloads:** full source chunk records are included in the response, each containing the original text, section title, context tag, and source label.
3. **Optional LLM synthesis:** when enabled, an LLM generates a natural language answer from the assembled evidence context. The LLM receives only retrieved facts—it does not draw on parametric knowledge.
4. **Fallback behavior:** when evidence is insufficient, the system returns a structured but limited response rather than generating unsupported content.

The query type label is included in every response, providing transparency about which retrieval path was taken.

9 Experimental Setup

9.1 Evaluation Framework

We evaluate the system using a benchmark harness that compares three retrieval configurations on a set of mental health questions with reference answers.

9.2 Systems Under Comparison

1. **Vector Baseline:** plain vector retrieval using sentence-transformers (`all-MiniLM-L6-v2` [13]) with top-$k = 8$ chunks, answered via context synthesis.
2. **Vector Reranked:** vector retrieval followed by cross-encoder reranking (`ms-marco-MiniLM-L-6-v` with top-$k = 5$ reranked chunks, answered via context synthesis.
3. **GraphRAG:** the full graph-first pipeline using `hybrid_query` with community summaries and optional LLM answer synthesis.

The vector retrieval stack includes a TF-IDF-based lexical fallback when sentence-transformer models are unavailable, and a token-overlap-based reranking fallback when the cross-encoder is unavailable, ensuring the benchmark runs in any environment.

9.3 Evaluation Metrics

When the RAGAS framework [4] is available with an OpenAI API key, four metrics are computed using `gpt-4o-mini` as the judge LLM and `text-embedding-3-small` for embeddings:

1. **Answer Relevancy:** semantic similarity between the generated answer and the reference.
2. **Context Precision:** fraction of retrieved context that is relevant to the ground truth.
3. **Context Recall:** fraction of ground-truth information captured by retrieved context.
4. **Faithfulness:** fraction of generated claims supported by retrieved context.

When RAGAS is unavailable, the harness falls back to deterministic proxy metrics based on token overlap (F1 for answer relevancy, precision/recall over context tokens, and answer-in-context ratio for faithfulness). The benchmark enforces node-hash compatibility between node and graph checkpoints, and saves per-system prediction outputs as JSONL for post-hoc analysis.

10 Results and Discussion

10.1 System Behavior Observations

We report qualitative and structural observations from the implemented system, as the current evaluation focuses on system behavior validation rather than large-scale quantitative benchmarking.

Query routing accuracy. The rule-based classification correctly routes the observed query types: condition-specific questions trigger forward lookup, symptom descriptions trigger reverse symptom matching, comparison phrasing triggers parallel condition expansion, and crisis language triggers safety short-circuit. Integration tests validate these routing behaviors across prevalence queries, natural symptom phrasing, typo tolerance, and session follow-up.

Schema enforcement. The constrained extraction ontology (11 entity types, 13 relations, 25 valid triplet patterns) combined with post-extraction cleaning effectively filters extraction noise. Garbage entity detection, placeholder filtering, and strict triplet validation provide multiple layers of quality control.

Repair effectiveness. Targeted repair and curated payload injection successfully address known factual gaps. Regression tests verify that depression prevalence data, GAD differential diagnosis, PTSD differential diagnosis, and autism treatment coverage are correctly represented in the graph after repair.

Provenance completeness. Every query response includes citation strings and full source chunk payloads, enabling manual verification of any claim against its original source text. The citation chain is maintained from source ingestion through extraction, repair, and query-time assembly.

Safety behavior. The crisis detection layer correctly triggers on explicit self-harm language and suppresses false positives for informational queries about crisis topics. When triggered, graph retrieval is bypassed entirely, and emergency resources are returned.

10.2 Architectural Trade-offs

Graph-first vs. hybrid retrieval. The current system uses graph-first retrieval without vector search in the query path. This ensures all returned evidence has explicit graph provenance, but may miss relevant information that exists in source text but was not captured during graph extraction. A hybrid approach combining graph and vector retrieval could improve recall.

Rule-based vs. learned routing. Query classification uses keyword matching, regex patterns, and fuzzy entity matching rather than a trained classifier. This approach is transparent and deterministic but less flexible for unexpected phrasings. The symptom phrase alias table (22 colloquial-to-clinical mappings) partially mitigates this limitation.

11 Limitations and Future Work

Rule-based query routing. The current classification relies on pattern matching and heuristics, which may fail on novel or implicit phrasings. A learned classifier trained on clinical query data could improve coverage.

LLM extraction quality. Graph quality depends on the extraction LLM's adherence to schema constraints and prompt instructions. Extraction errors propagate through the pipeline despite multi-layer cleaning.

Safety coverage. The regex-based crisis detection is effective for explicit self-harm language but may miss nuanced, implicit, or multilingual crisis signals. Integration with a trained safety classifier could improve recall.

Evaluation scale. The current evaluation uses proxy metrics and targeted regression tests. A larger-scale evaluation with clinician-annotated queries and standardized medical QA benchmarks would strengthen quantitative claims.

Hybrid retrieval. Integrating vector retrieval alongside graph-first retrieval could capture information present in source text but absent from the extracted graph, improving recall without sacrificing provenance.

12 Conclusion

We present a clinical GraphRAG framework for mental health question answering that prioritizes evidence grounding, safety, and provenance over unconstrained generation. The system constructs a knowledge graph from authoritative clinical sources (WHO, NIMH, DSM-5) using schema-constrained extraction with a fixed ontology of 11 entity types, 13 relation types, and 25 valid triplet patterns. Post-extraction cleaning, normalization, targeted repair, and curated payload injection ensure graph quality.

At query time, a safety-first routing layer detects crisis signals before any retrieval occurs. In-scope queries are processed through graph-first multi-path retrieval, returning structured answers with full citation chains and source chunk payloads. The system supports forward lookup, reverse symptom matching, condition comparison, and community-level contextual grounding.

The end-to-end pipeline is fully checkpointed and reproducible, with integration tests validating routing behavior, answer quality constraints, and safety logic. A benchmark harness enables systematic comparison of vector, reranked vector, and GraphRAG retrieval configurations.

This work demonstrates that a graph-grounded, safety-aware RAG architecture can provide auditable and traceable clinical question answering, establishing a practical foundation for further evaluation and hybrid retrieval extensions in the mental health domain.

Acknowledgments

We thank the World Health Organization, the National Institute of Mental Health, and the developers of the DSM-5 for providing the authoritative clinical sources used in this work.

References

[1] American Psychiatric Association. *Diagnostic and Statistical Manual of Mental Disorders (DSM-5)*. American Psychiatric Publishing, 5th edition, 2013.

[2] Adrien Barbaresi. Trafilatura: A web scraping library and command-line tool for text discovery and extraction. In *Proceedings of the Joint Conference of the 59th Annual Meeting of the Association for Computational Linguistics*, pages 122–131, 2021.

[3] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[4] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2024.

[5] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-augmented language model pre-training. *arXiv preprint arXiv:2002.10157*, 2020.

[6] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, et al. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.

[7] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[9] National Institute of Mental Health. Mental health information. https://www.nimh.nih.gov/health/topics, 2024. Accessed: 2025.

[10] Rodrigo Nogueira, Zhiying Jiang, and Jimmy Lin. Document ranking with a pre-trained sequence-to-sequence model. *arXiv preprint arXiv:2003.06713*, 2020.

[11] Harsha Nori, Yin Tat Lee, Sheng Zhang, Dean Carignan, Richard Edgar, Nicolo Fusi, Nicholas King, Jonathan Larson, Yuanzhi Li, Weishung Liu, et al. Can generalist foundation models outcompete special-purpose tuning? case study in medicine. *arXiv preprint arXiv:2311.16452*, 2023.

[12] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiawei Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.

[13] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019.

[14] Karan Singhal, Shekoofeh Azizi, Tao Tu, S Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, et al. Large language models encode clinical knowledge. *Nature*, 620(7972):172–180, 2023.

[15] V. A. Traag, L. Waltman, and N. J. van Eck. From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports*, 9(1):5233, 2019.

[16] World Health Organization. Mental health. https://www.who.int/health-topics/mental-health, 2024. Accessed: 2025.